ST>>>>>X

Collège Sainte-Croix
Kollegium Heilig Kreuz

L'unité de commande

Option Complémentaire Informatique

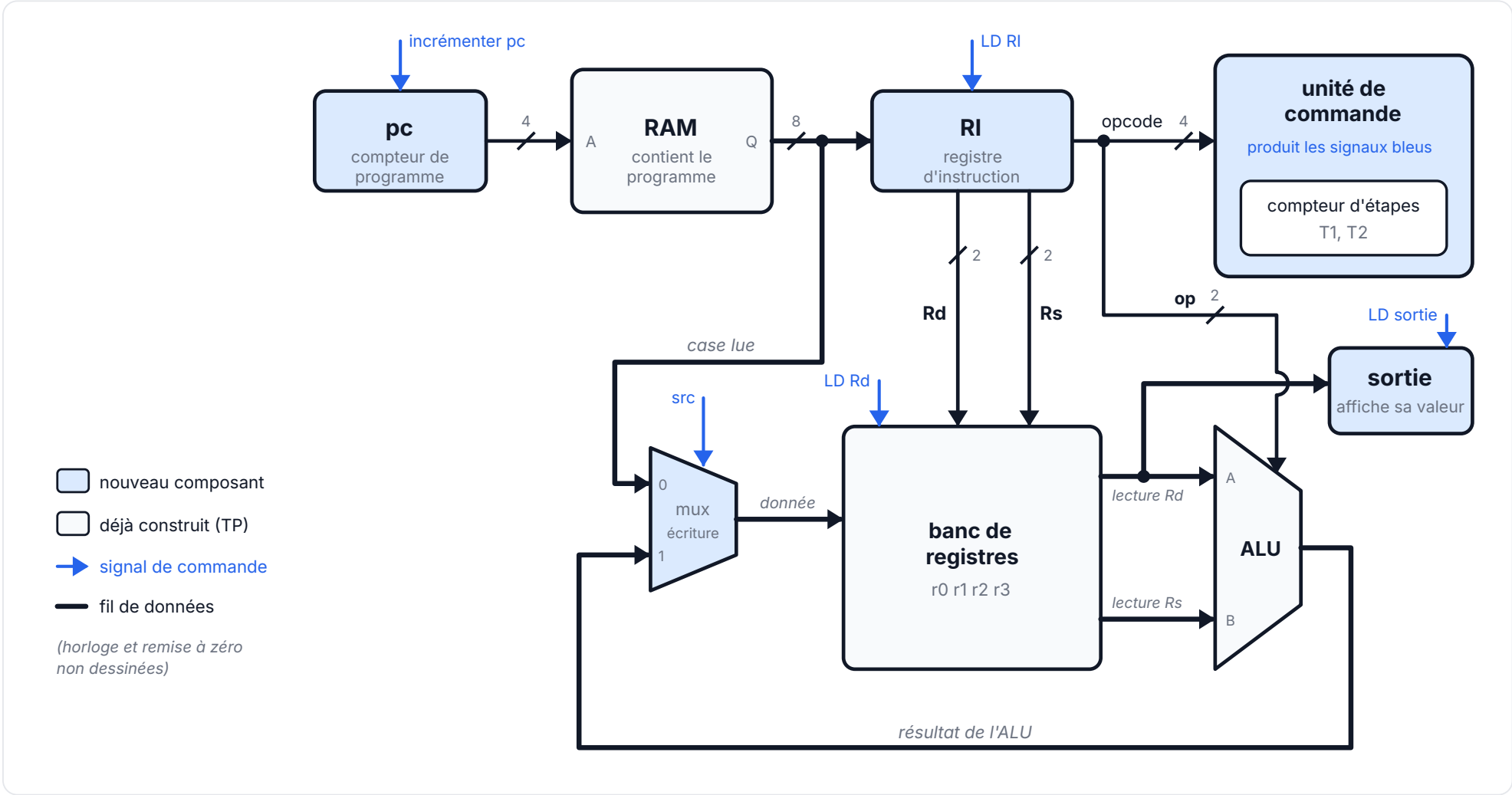
π

Où en sommes-nous ?

- Nous avons construit l'ALU, le banc de registres et la RAM, qui contient déjà le programme
- Mais c'est nous qui actionnons tout à la main :
  - choisir les registres Rd et Rs
  - choisir l'opcode
  - changer l'adresse de la RAM

Objectif : remplacer ces interrupteurs par des circuits pour que le processeur travaille **tout seul** à partir de l'instruction dans la RAM

Le processeur complet



Les nouveaux composants

composant	rôle	dans Logix	TP

Dans Logix

pour faire…	dans la palette	réglages (propriétés)
un compteur	Compteur N-bit	largeur : 4 bits (pc), 1 bit (étapes)
un registre	Registre N-bit	largeur : 8 bits
un champ de bits	Tranche	Bit haut et Bit bas du champ
un multiplexeur	Multiplexeur	Bits de sélection, Largeur des données
un décodeur	Décodeur	Largeur d'entrée : 1 bit ou 4 bits
portes ET, OU	AND, OR	(rien à régler)

Front montant = 2 clics sur l'horloge : 0 → 1 (le front), puis 1 → 0

reset = mettre l'entrée reset à 1, puis la remettre à 0

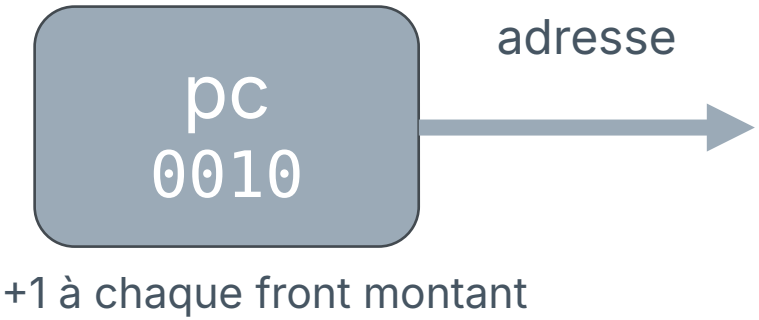
Partie 1 : chercher les instructions

Le compteur de programme (TP 1) et le registre d’instruction (TP 2)



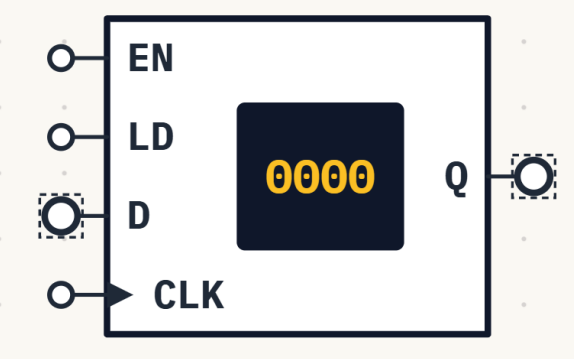
Le compteur de programme (pc)

- Un compteur : un registre qui ajoute 1 à sa propre valeur à chaque front montant
- Il contient l'adresse de la prochaine instruction à lire en mémoire
- Son entrée LD (D non branchée) le remet à 0 **tout de suite**, sans attendre de front montant



adresse	contenu
0000	0001 0000
0001	0000 1101
0010	0001 0100
0011	0000 0010

Le Compteur N-bit dans Logix



nom	de l'anglais	rôle
EN	enable : activer	si 1 : +1 à chaque front montant
LD	load : charger	si 1 : prend tout de suite D
D	data : donnée	la valeur à charger (0 si libre)
CLK	clock : horloge	ses fronts montants font avancer
Q	(sortie)	la valeur actuelle du compteur

TP 1 : le compteur de programme

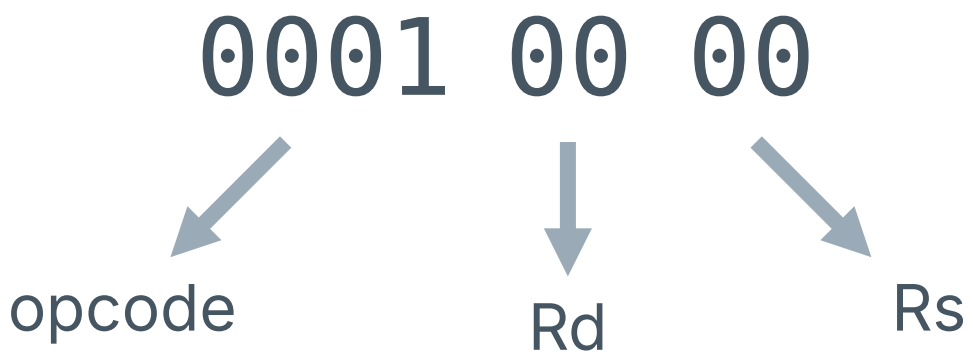
› Rendez-vous sur la page « Unité de commande » et réalisez la première partie du TP

A la fin, vérifiez :

- reset (1 puis 0) : pc = `0000`, la RAM affiche `0001 0000`
- incrémenter pc à 1, puis quelques fronts montants : la RAM affiche les cases une à une

Le registre d'instruction (RI)

- Quand le pc avance, l'instruction présentée par la RAM change, mais il faut **mémoriser** l'instruction un moment
- Le RI est un registre de 8 bits relié à la sortie de la RAM : il capture l'instruction lue quand LD = 1



des Tranches découpent RI
(bits 7 à 4, 3 et 2, 1 et 0)

Rd et Rs vont directement au banc : c'est l'instruction qui choisit les registres

TP 2 : le registre d'instruction

- Placez un **Registre N-bit** de 8 bits nommé **RI** : D (donnée) ← sortie Q de la RAM, CLK ← horloge, LD (charger) ← nouvelle entrée **LD RI**
- Trois **Tranches** sur la sortie Q de RI : opcode (bits 7 à 4), Rd (3 et 2), Rs (1 et 0)
- Tranche Rd → décodeur du banc **et** sel du mux lecture Rd ; Tranche Rs → sel du mux lecture Rs. Supprimez les entrées Rd et Rs

Vérifiez

- reset, puis incrémenter pc = 1 et LD RI = 1
- 1er front montant : RI = `0001 0000` (LOAD r0)
- 2e front montant : RI = `0000 1101`… la valeur 13 ! Il faudra mettre LD RI à 1 **au bon moment**

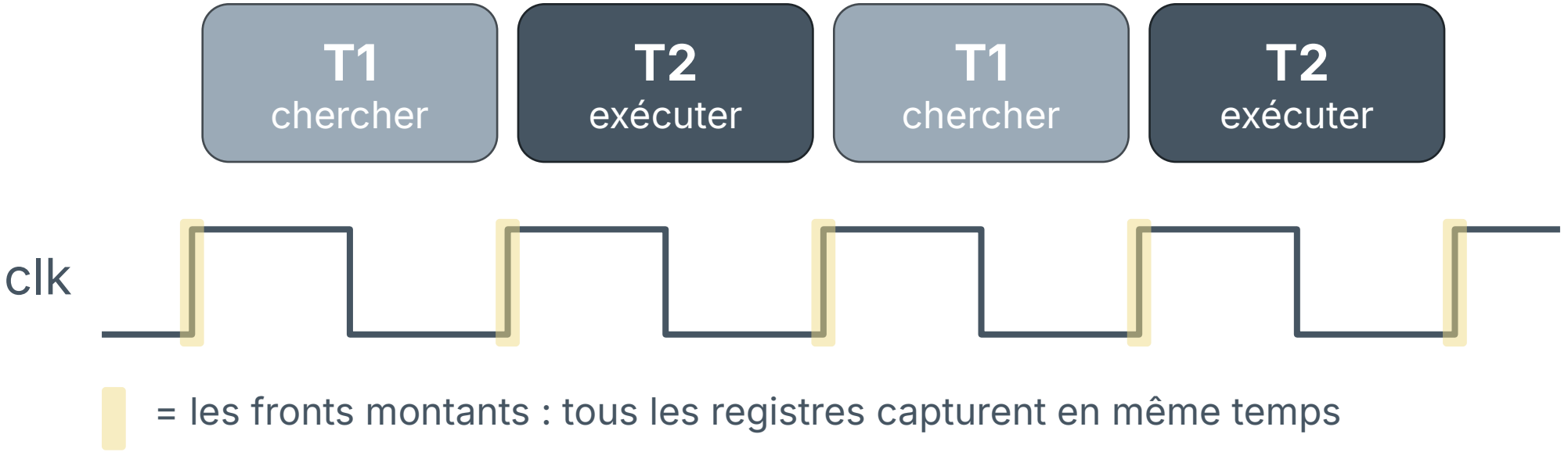
Partie 2 : exécuter les instructions

Deux étapes, le multiplexeur d’écriture et le registre de sortie (TP 3)

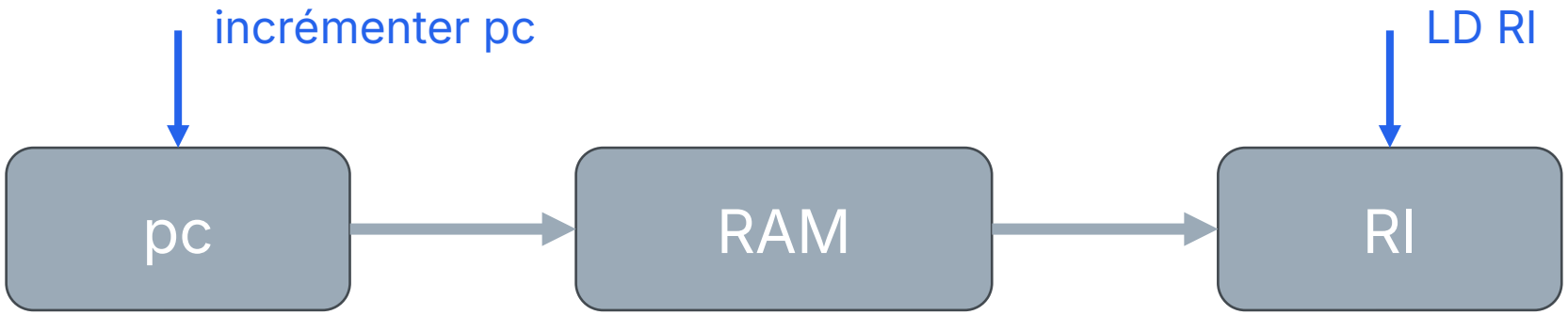


Une instruction en deux étapes

- Un registre ne capture une valeur qu'au front montant de l'horloge
- Une instruction se déroule donc en étapes, une par front montant :



Chercher (T1)



- La RAM présente en permanence la case dont l'adresse est dans pc
- Au front montant qui termine T1, RI capture cette instruction **et** pc avance de 1
- Pas de conflit : au front montant, tous les registres capturent **en même temps** la valeur présente juste avant le front

Et l'étape « décoder » ?

Notre processeur

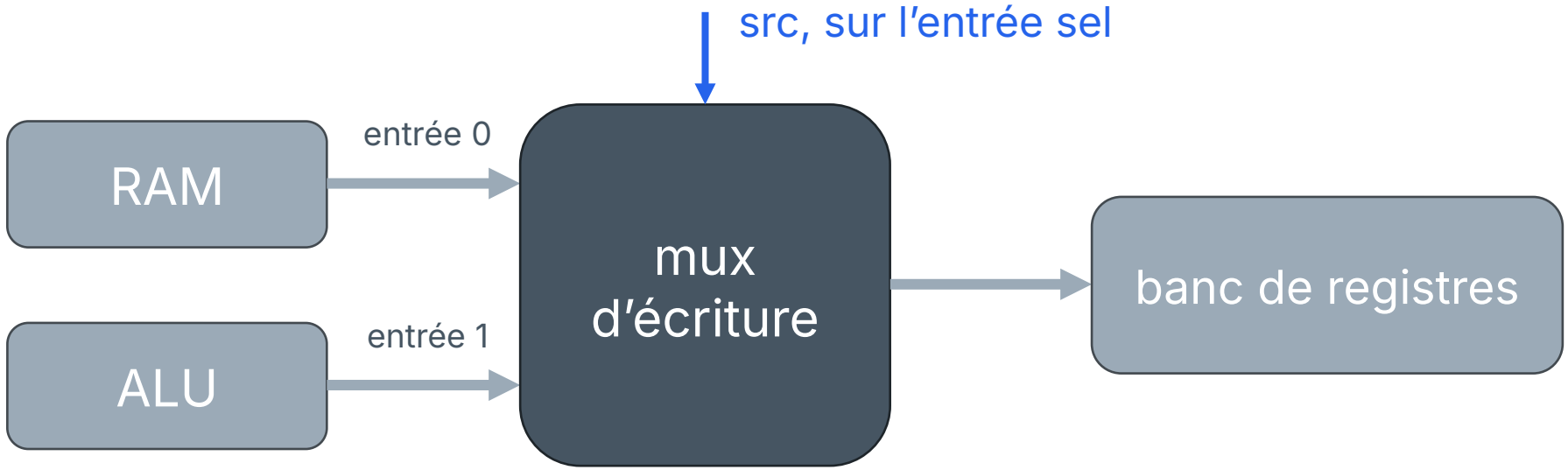
- 2 étapes seulement : chercher, puis exécuter
- Décoder = découper 8 bits en champs
- Des Tranches et un décodeur le font **en continu**, sans attendre de front montant

Un vrai processeur

- Une vraie étape de décodage : des centaines d'instructions, de 1 à 15 octets (Intel, AMD)
- Travail **à la chaîne** (pipeline) : pendant qu'une instruction s'exécute, la suivante est décodée, et celle d'après est cherchée

Le multiplexeur d'écriture

- Pendant T2, LOAD et les calculs rangent une valeur dans Rd. Laquelle ?
  - LOAD : la valeur lue dans la **mémoire**
  - ADD, SUB, AND, OR : le **résultat de l'ALU**



src et op : aucune porte !

opcode	instruction	effet
`0101`	SUB Rd, Rs	Rd reçoit Rd - Rs

Grâce au choix des opcodes, src et op se lisent directement dans l'instruction



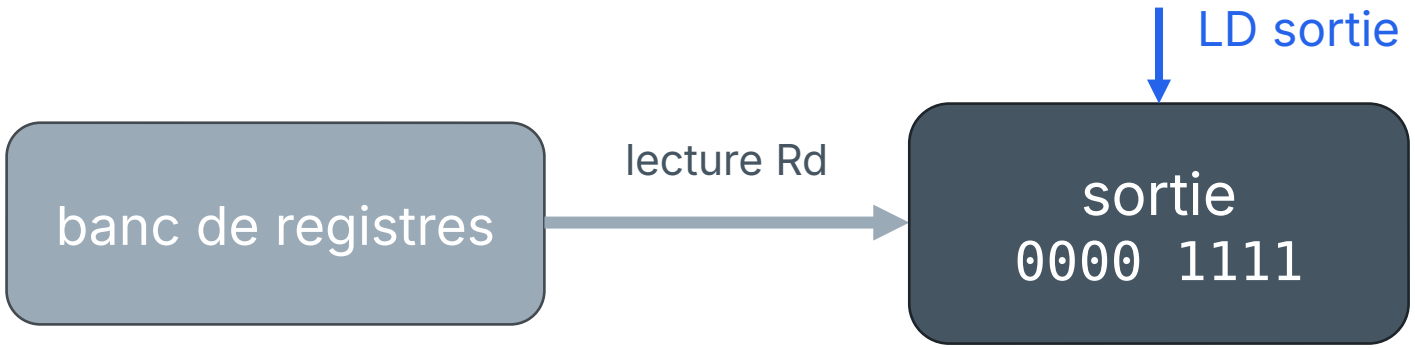
commence par 01 : calcul
src = bit 6 de RI

op de l'ALU : 01 = soustraction
op = bits 5 et 4 de RI

Codes de l'ALU : `00` addition, `01` soustraction, `10` ET, `11` OU

Le registre de sortie

- OUT Rd montre le contenu du registre Rd
- Le banc le lit déjà en permanence : c'est la lecture Rd, qui part vers l'entrée A de l'ALU
- Un registre de sortie de 8 bits capture la lecture Rd quand LD sortie = 1 : c'est l'écran du processeur



Les signaux de chaque étape

Les signaux à 1 dépendent de l'étape et de l'instruction (les autres valent 0) :

étape	instruction	signaux à 1
T1	toutes	
T2	LOAD Rd, valeur	
T2	ADD, SUB, AND, OR	
T2	OUT Rd	
T2	STOP	

src et op, eux, se règlent tout seuls

Le cas de LOAD

LOAD occupe 2 cases de la mémoire : l'instruction, puis la valeur à charger

pc →

adresse	contenu	signification
`0000`	`0001 0000`	LOAD r0, …
`0001`	`0000 1101`	valeur : 13
`0010`	`0001 0100`	LOAD r1, …
`0011`	`0000 0010`	valeur : 2

T1 : RI capture `0001 0000` (LOAD r0) et pc avance à `0001`

Le cas de LOAD

LOAD occupe 2 cases de la mémoire : l'instruction, puis la valeur à charger

pc →

adresse	contenu	signification
`0000`	`0001 0000`	LOAD r0, …
`0001`	`0000 1101`	valeur : 13
`0010`	`0001 0100`	LOAD r1, …
`0011`	`0000 0010`	valeur : 2

T2 : pc pointe déjà sur la valeur ! r0 capture `0000 1101`,
et pc avance encore (`0010`) pour sauter par-dessus la valeur

TP 3 : jouez l'unité de commande

- **Multiplexeur** mux d'écriture (sélection 1 bit, données 8 bits) : entrée 0 ← RAM, entrée 1 ← ALU, sortie → D des registres
- Deux **Tranches** sur RI : **calcul** (bit 6) → sel du mux, **op** (bits 5 et 4) → op de l'ALU
- **Registre N-bit** sortie (pas une Sortie !) de 8 bits : D ← lecture Rd, LD ← entrée **LD sortie**
- Supprimez les entrées donnée et op, et renommez LD en **LD Rd**

Il reste 4 interrupteurs : incrémenter pc, LD RI, LD Rd et LD sortie
L'unité de commande, c'est vous !

TP 3 : à vous de jouer !

front	étape	instruction	incrémenter pc	LD RI	LD Rd	LD sortie
1	T1	LOAD r0, 13				
2	T2	LOAD r0, 13				
3	T1	LOAD r1, 2				
4	T2	LOAD r1, 2				
5	T1	ADD r0, r1				
6	T2	ADD r0, r1				
7	T1	OUT r0				
8	T2	OUT r0				

Réglez pendant l'étape, puis le front montant qui la termine
Au 8e front, la sortie affiche `0000 1111`

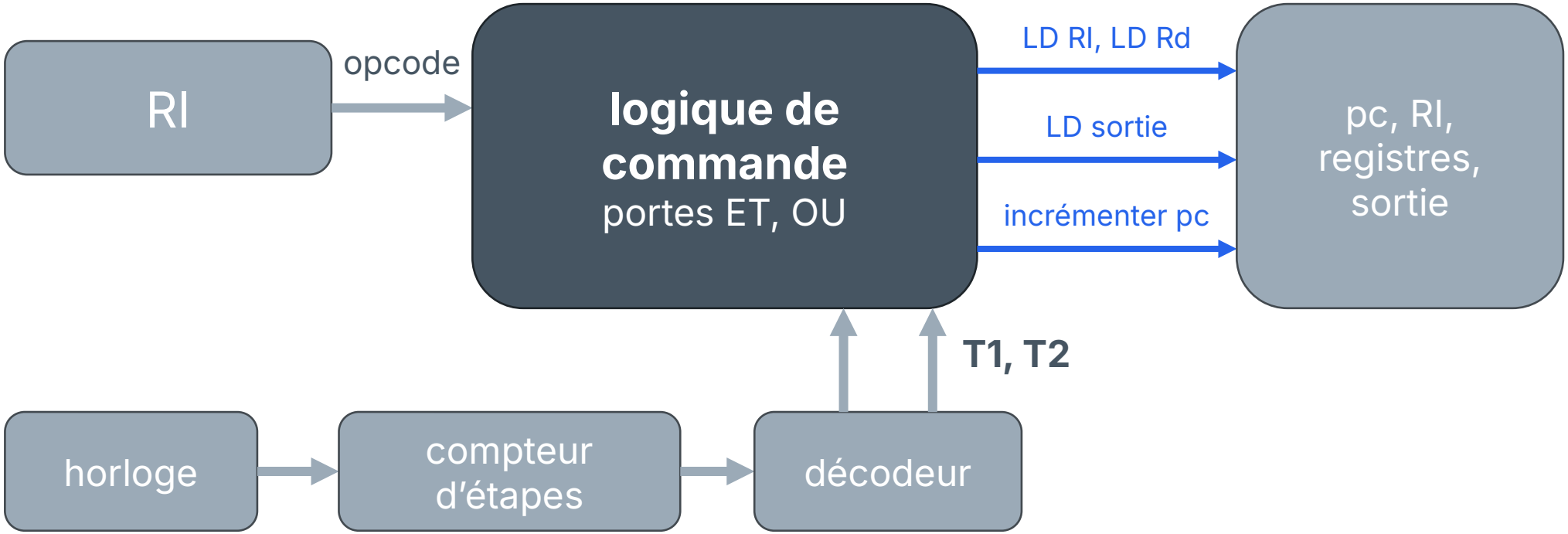
Partie 3 : automatiser

L’unité de commande (TP 4)



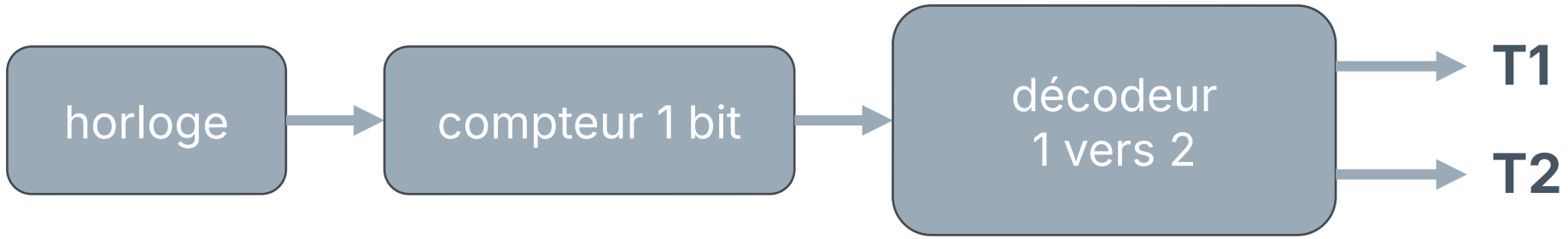
L'unité de commande

Elle fait ce que vous faisiez à la main : combiner l'opcode et l'étape pour produire les signaux



Le compteur d'étapes

- Un compteur de 1 bit, suivi d'un décodeur 1 vers 2
- Le décodeur n'allume que la sortie dont le numéro est son entrée : sortie 0 = T1, sortie 1 = T2
- À chaque front montant, on passe à l'autre étape : T1, T2, T1, T2…
- Chaque instruction dure donc toujours **2 étapes**



Reconnaître l'instruction

- Un décodeur 4 vers 16 sur l'opcode donne une ligne par instruction
- Les instructions de calcul commencent toutes par 01 : le bit 6 de RI suffit, on l'appelle **calcul**

opcode	instruction	ligne utilisée
0001	LOAD Rd, valeur	LOAD (sortie 1)
0010	OUT Rd	OUT (sortie 2)
0100	ADD Rd, Rs	**calcul**
0101	SUB Rd, Rs	**calcul**
0110	AND Rd, Rs	**calcul**
0111	OR Rd, Rs	**calcul**

Quand chaque signal vaut-il 1 ?

- Il suffit de relire le tableau des signaux de chaque étape, signal par signal
- Exemple : incrémenter pc vaut 1…
  - à l'étape T1, quelle que soit l'instruction
  - à l'étape T2, quand l'instruction est LOAD

incrémenter pc = T1 OU (LOAD ET T2)

Quand chaque signal vaut-il 1 ?

signal	relié à	vaut 1 quand
LD RI	LD du registre d'instruction	
incrémenter pc	EN du compteur de programme	
LD Rd	LD du banc de registres	
LD sortie	LD du registre de sortie	
src	sélection du mux d'écriture	
op	op de l'ALU	

Au total : 3 portes ET et 2 portes OU

Pourquoi tester aussi T2 ?

- Pendant T1, RI contient encore l'instruction **précédente** (au départ `0000 0000`) : un signal qui dépend de l'instruction teste donc toujours aussi T2
- src et op ne servent qu'au moment où Rd capture : ils peuvent dépendre de l'instruction seule

TP 4 : l'unité de commande (1/2)

- **Compteur d'étapes** : Compteur N-bit de **1 bit** : CLK ← horloge, LD ← reset, EN ← nouvelle entrée **avancer** (mise à 1)
- Décodeur (largeur d'entrée 1 bit) : sortie 0 = **T1**, sortie 1 = **T2**, avec deux lampes T1 et T2
- **LD RI = T1**. Reliez T1 à LD de RI et supprimez l'interrupteur LD RI

Vérifiez

- reset : T1 est allumée ; à chaque front montant, T1 et T2 alternent

TP 4 : l'unité de commande (2/2)

- **Décodeur** 4 vers 16 sur la Tranche opcode : sortie 1 = LOAD, sortie 2 = OUT
- Avec 3 **AND** et 2 **OR** : incrémenter pc, LD Rd et LD sortie, chacun là où arrivait son interrupteur
- reset, puis fronts montants. Au 1er : T2 allumée, RI = `0001 0000`, pc = `0001`

Au 8e front montant, la sortie affiche `0000 1111` = 15
Vous venez de construire un ordinateur !

Exécutons le programme test

adresse	contenu	instruction
0000	0001 0000	LOAD r0, …
0001	0000 1101	(valeur : 13)
0010	0001 0100	LOAD r1, …
0011	0000 0010	(valeur : 2)
0100	0100 0001	ADD r0, r1
0101	0010 0000	OUT r0
0110	0000 0000	STOP

Au départ :

pc = 0000

RI = 0000 0000

r0 à r3 = 0000 0000

À vous de jouer

- Pour les 8 premiers fronts montants : pc après le front (4 bits), registre mis à jour et sa valeur

Exécutons le programme test

front	étape	pc après	registre mis à jour	sa valeur
1	T1			
2	T2			
3	T1			
4	T2			
5	T1			
6	T2			
7	T1			
8	T2			

Front 6 : ADD n'occupe qu'une case, pc n'avance pas.
Front 9 : RI reçoit STOP ; ensuite, plus aucun registre ne change.

Les signaux pendant une instruction

À vous de jouer

- SUB r1, r0 à l'étape T2 : que vaut chaque signal de commande ?

```
0101 01 00
```

SUB r1, r0

signal	valeur
LD RI	
incrémenter pc	
LD Rd	
LD sortie	
src	
op	